

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

```
1 df = pd.read_csv("/content/IMDB Dataset.csv")
2
3 df.head()
```

	review	sentiment	
0	One of the other reviewers has mentioned that ...	positive	
1	A wonderful little production. The...	positive	
2	I thought this was a wonderful way to spend ti...	positive	
3	Basically there's a family where a little boy ...	negative	
4	Petter Mattei's "Love in the Time of Money" is...	positive	

Next steps: [New interactive sheet](#)

```
1 print("Total rows:", len(df))
2 print("Unique sentiments:", df['sentiment'].unique())
3 print("Class distribution:")
4 print(df['sentiment'].value_counts())
```

```
Total rows: 50000
Unique sentiments: ['positive' 'negative']
Class distribution:
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

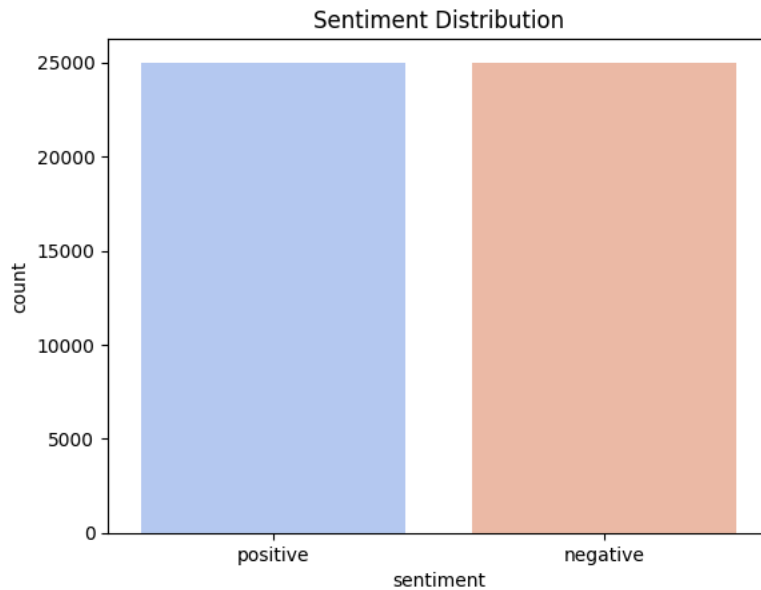
✓ class balance visulization

```
1 sns.countplot(data=df, x="sentiment", palette="coolwarm")
2 plt.title("Sentiment Distribution")
3 plt.show()
```

```
/tmp/ipython-input-3163056628.py:1: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `1`

```
sns.countplot(data=df, x="sentiment", palette="coolwarm")
```



✓ WordCloud Visualization

```
1 !pip install nltk spacy wordcloud tqdm
```

[Show hidden output](#)

```
1 from wordcloud import WordCloud
2
3 # Positive reviews wordcloud
4 text_pos = " ".join(df[df['sentiment']=='positive']['review'].astype(str))
5 wordcloud_pos = WordCloud(width=800, height=400, background_color="white").generate(text_pos)
6
7 plt.figure(figsize=(10,5))
8 plt.imshow(wordcloud_pos, interpolation="bilinear")
9 plt.axis("off")
10 plt.title("Positive Reviews WordCloud")
11 plt.show()
12
13 # Negative reviews wordcloud
14 text_neg = " ".join(df[df['sentiment']=='negative']['review'].astype(str))
15 wordcloud_neg = WordCloud(width=800, height=400, background_color="black", colormap="Reds").gener
16
17 plt.figure(figsize=(10,5))
18 plt.imshow(wordcloud_neg, interpolation="bilinear")
19 plt.axis("off")
20 plt.title("Negative Reviews WordCloud")
21 plt.show()
22
```



```

3
4     doc = nlp(text)
5
6     tokens = [token.lemma_ for token in doc if token.text not in stop_words and len(token.text) >
7
8     return " ".join(tokens)
9

```

```

1 from tqdm import tqdm
2 tqdm.pandas()
3
4 df["clean_review"] = df["review"].astype(str).progress_apply(preprocess_text)
5
6 df.head()

```

100%|██████████| 50000/50000 [34:08<00:00, 24.40it/s]

	review	sentiment	clean_review
0	One of the other reviewers has mentioned that ...	positive	one reviewer mention watch episode hook right ...
1	A wonderful little production. The...	positive	wonderful little production filming technique ...
2	I thought this was a wonderful way to spend ti...	positive	think wonderful way spend time hot summer week...
3	Basically there's a family where a little boy ...	negative	basically family little boy jake think zombie ...
4	Petter Mattei's "Love in the Time of Money" is...	positive	petter mattei love time money visually stunnin...

Next steps: [New interactive sheet](#)

```
1 "Original Review:\n", df["review"].iloc[0]
```

```

('Original Review:\n',
 "One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is
 exactly what happened with me.<br /><br />The first thing that struck me about Oz was its brutality and unflinching scenes of
 violence, which set in right from the word GO. Trust me, this is not a show for the faint hearted or timid. This show pulls no
 punches with regards to drugs, sex or violence. Its is hardcore, in the classic use of the word.<br /><br />It is called OZ as
 that is the nickname given to the Oswald Maximum Security State Penitentiary. It focuses mainly on Emerald City, an experimental
 section of the prison where all the cells have glass fronts and face inwards, so privacy is not high on the agenda. Em City is
 home to many..Aryans, Muslims, gangstas, Latinos, Christians, Italians, Irish and more....so scuffles, death stares, dodgy
 dealings and shady agreements are never far away.<br /><br />I would say the main appeal of the show is due to the fact that it
 goes where other shows wouldn't dare. Forget pretty pictures painted for mainstream audiences, forget charm, forget romance...OZ
 doesn't mess around. The first episode I ever saw struck me as so nasty it was surreal, I couldn't say I was ready for it, but as
 I watched more, I developed a taste for Oz, and got accustomed to the high levels of graphic violence. Not just violence, but
 injustice (crooked guards who'll be sold out for a nickel, inmates who'll kill on order and get away with it, well mannered,
 middle class inmates being turned into prison bitches due to their lack of street skills or prison experience) Watching Oz, you
 may become comfortable with what is uncomfortable viewing....thats if you can get in touch with your darker side.")

```

```
1 "\nCleaned Review:\n", df["clean_review"].iloc[0]
```

```

('\nCleaned Review:\n',
 'one reviewer mention watch episode hook right exactly happen first thing strike brutality unflinche scene violence set right
 word trust show faint hearted timid show pull punch regard drug sex violence hardcore classic use word call nickname give oswald
 maximum security state penitentiary focus mainly emerald city experimental section prison cell glass front face inward privacy high
 agenda city home many aryan muslims gangstas latinos christians italian irish scuffle death stare dodgy dealing shady agreement
 never far away would say main appeal show due fact go show dare forget pretty picture paint mainstream audience forget charm
 forget romance mess around first episode ever see strike nasty surreal say ready watch develop taste get accustom high level
 graphic violence violence injustice crook guard sell nickel inmate kill order get away well mannered middle class inmate turn
 prison bitch due lack street skill prison experience watch may become comfortable uncomfortable view get touch dark side')

```

✓ Review Length Distribution (Before vs After)

```

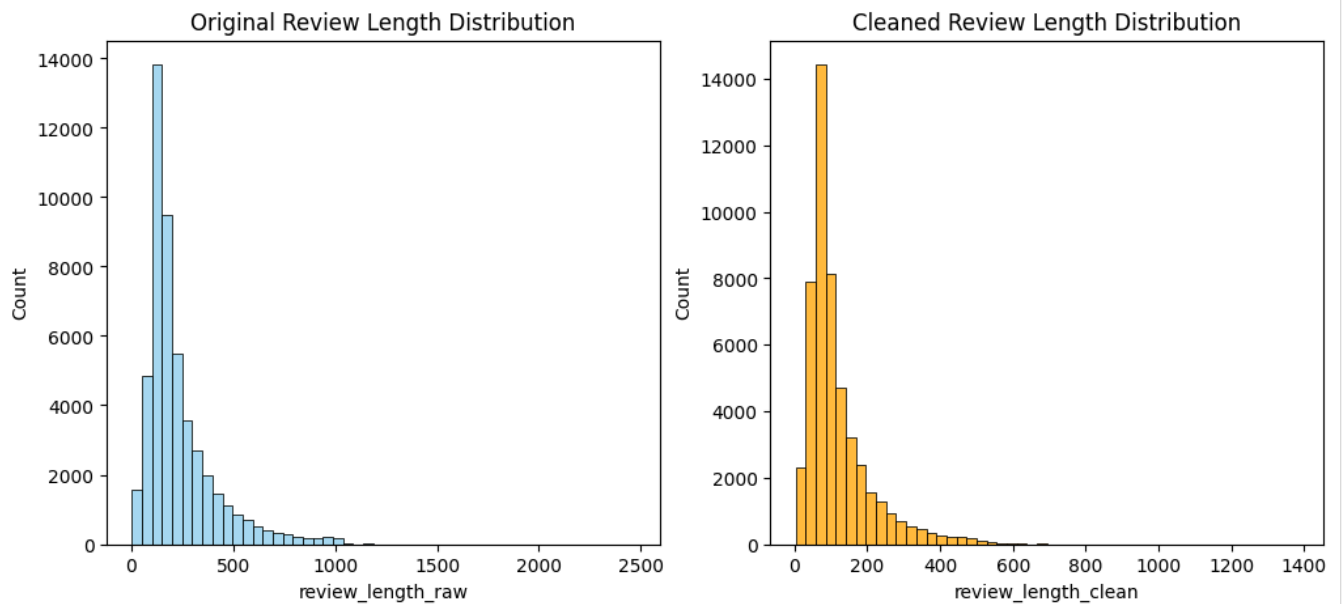
1 df["review_length_raw"] = df["review"].astype(str).apply(lambda x: len(x.split()))
2
3 df["review_length_clean"] = df["clean_review"].apply(lambda x: len(x.split()))
4
5 plt.figure(figsize=(12,5))
6
7 plt.subplot(1,2,1)
8 sns.histplot(df["review_length_raw"], bins=50, color="skyblue")
9

```

```

9 plt.title("Original Review Length Distribution")
10
11 plt.subplot(1,2,2)
12 sns.histplot(df["review_length_clean"], bins=50, color="orange")
13 plt.title("Cleaned Review Length Distribution")
14
15 plt.show()

```



```

1 from sklearn.model_selection import train_test_split
2
3 X = df["clean_review"]
4 y = df["sentiment"]
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42, stratify=y
8 )
9
10 print("Train size:", len(X_train))
11 print("Test size:", len(X_test))
12

```

Train size: 40000
Test size: 10000

✓ TF-IDF Vec

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 tfidf = TfidfVectorizer(
4     max_features=5000,
5     ngram_range=(1,2)
6 )
7
8 X_train_tfidf = tfidf.fit_transform(X_train)
9
10 X_test_tfidf = tfidf.transform(X_test)
11
12 print("TF-IDF train shape:", X_train_tfidf.shape)
13 print("TF-IDF test shape:", X_test_tfidf.shape)
14

```

```
TF-IDF train shape: (40000, 5000)
TF-IDF test shape: (10000, 5000)
```

```
1 from sklearn.linear_model import LogisticRegression
2
3 log_reg = LogisticRegression(max_iter=1000)
4
5 log_reg.fit(X_train_tfidf, y_train)
6
7 y_pred = log_reg.predict(X_test_tfidf)
```

```
1 from sklearn.metrics import classification_report, confusion_matrix
2
3 print("Logistic Regression Evaluation Report:\n")
4 print(classification_report(y_test, y_pred))
```

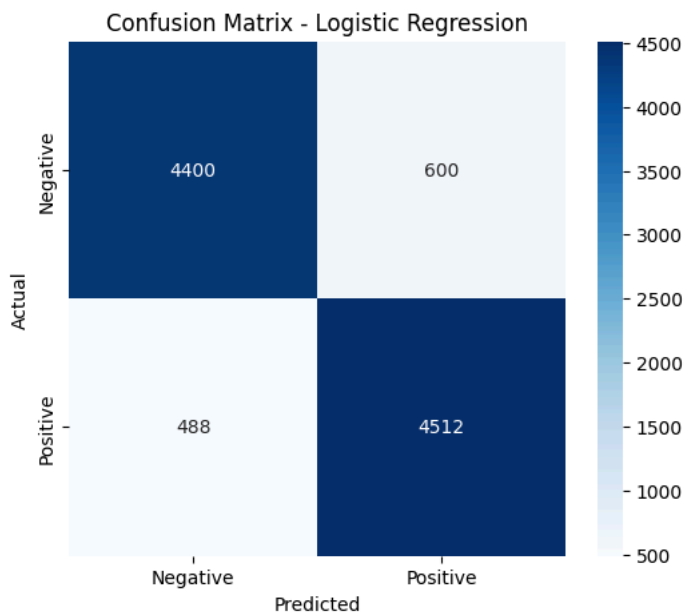
Logistic Regression Evaluation Report:

	precision	recall	f1-score	support
negative	0.90	0.88	0.89	5000
positive	0.88	0.90	0.89	5000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

```
1 from sklearn.model_selection import cross_val_score
2
3 cv_scores = cross_val_score(LogisticRegression(max_iter=1000), X_train_tfidf, y_train, cv=5, scoring='accuracy')
4 print("Cross Validation Scores:", cv_scores)
5 print("Mean CV Accuracy:", np.mean(cv_scores))
```

Cross Validation Scores: [0.89425 0.881 0.88925 0.88175 0.886125]
Mean CV Accuracy: 0.8864749999999999

```
1 cm = confusion_matrix(y_test, y_pred)
2 plt.figure(figsize=(6,5))
3 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Negative","Positive"], yticklabels=["Negative","Positive"])
4 plt.xlabel("Predicted")
5 plt.ylabel("Actual")
6 plt.title("Confusion Matrix - Logistic Regression")
7 plt.show()
```



```

1 feature_names = tfidf.get_feature_names_out()
2 coefficients = log_reg.coef_[0]
3
4 top_pos = np.argsort(coefficients)[-20:]
5 top_neg = np.argsort(coefficients)[:20]
6
7 plt.figure(figsize=(12,6))

```

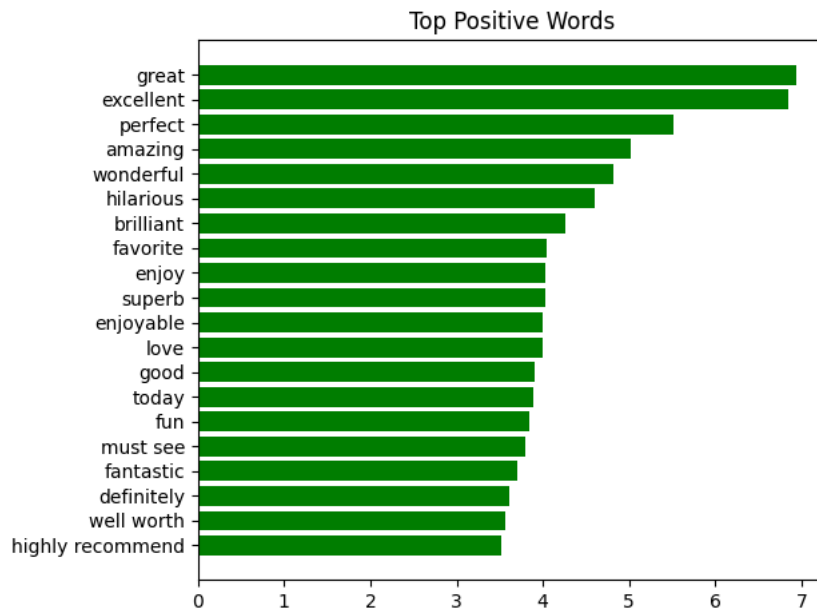
<Figure size 1200x600 with 0 Axes>

<Figure size 1200x600 with 0 Axes>

```

1 plt.subplot(1,1,1)
2 plt.barh(feature_names[top_pos], coefficients[top_pos], color="green")
3 plt.title("Top Positive Words")
4 plt.tight_layout()
5 plt.show()

```

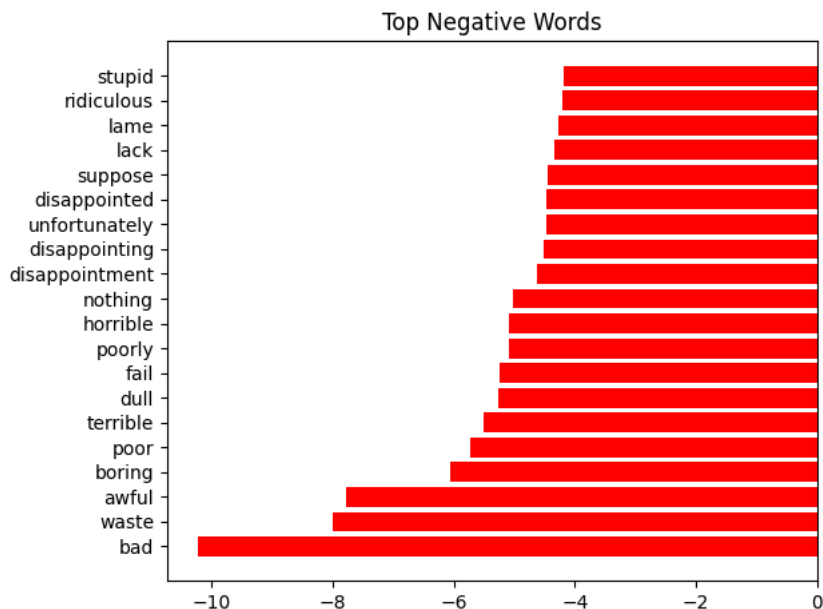


Cross Validation for LR

```

1 plt.subplot(1,1,1)
2 plt.barh(feature_names[top_neg], coefficients[top_neg], color="red")
3 plt.title("Top Negative Words")
4
5 plt.tight_layout()
6 plt.show()

```



✓ Deep Learning Model (LSTM)

```

1 from tensorflow.keras.preprocessing.text import Tokenizer
2 from tensorflow.keras.preprocessing.sequence import pad_sequences
3
4 MAX_WORDS = 10000
5 MAX_LEN = 200
6 tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token="<OOV>")
7 tokenizer.fit_on_texts(X_train)
8
9 X_train_seq = tokenizer.texts_to_sequences(X_train)
10 X_test_seq = tokenizer.texts_to_sequences(X_test)
11
12 X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding="post")
13 X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LEN, padding="post")
14
15 print("Training shape:", X_train_pad.shape)
16 print("Testing shape:", X_test_pad.shape)

```

Training shape: (40000, 200)
Testing shape: (10000, 200)

encoding

```

1 from sklearn.preprocessing import LabelEncoder
2
3 le = LabelEncoder()
4 y_train_enc = le.fit_transform(y_train)
5 y_test_enc = le.transform(y_test)

```

```

1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Bidirectional

```

```

1 from tensorflow.keras import Sequential
2 from tensorflow.keras.layers import Embedding, LSTM, Bidirectional, Dense, Dropout, Input
3
4 EMBEDDING_DIM = 128
5

```

```

6 model = Sequential([
7     Input(shape=(MAX_LEN,)),
8     Embedding(input_dim=MAX_WORDS, output_dim=EMBEDDING_DIM),
9     Bidirectional(LSTM(64, return_sequences=False)),
10    Dropout(0.5),
11    Dense(64, activation="relu"),
12    Dropout(0.5),
13    Dense(1, activation="sigmoid")
14 ])
15
16 model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
17 model.summary()
18

```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 200, 128)	1,280,000
bidirectional_2 (Bidirectional)	(None, 128)	98,816
dropout_4 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8,256
dropout_5 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 1)	65

Total params: 1,387,137 (5.29 MB)
 Trainable params: 1,387,137 (5.29 MB)

```

1 history = model.fit(
2     X_train_pad, y_train_enc,
3     validation_data=(X_test_pad, y_test_enc),
4     epochs=10,
5     batch_size=128,
6     verbose=1
7 )

```

```

Epoch 1/10
313/313 ————— 8s 25ms/step - accuracy: 0.9657 - loss: 0.1030 - val_accuracy: 0.8693 - val_loss: 0.4405
Epoch 2/10
313/313 ————— 7s 23ms/step - accuracy: 0.9729 - loss: 0.0862 - val_accuracy: 0.8756 - val_loss: 0.4481
Epoch 3/10
313/313 ————— 8s 25ms/step - accuracy: 0.9779 - loss: 0.0705 - val_accuracy: 0.8708 - val_loss: 0.4378
Epoch 4/10
313/313 ————— 8s 25ms/step - accuracy: 0.9788 - loss: 0.0667 - val_accuracy: 0.8526 - val_loss: 0.3954
Epoch 5/10
313/313 ————— 7s 24ms/step - accuracy: 0.9754 - loss: 0.0769 - val_accuracy: 0.8704 - val_loss: 0.5382
Epoch 6/10
313/313 ————— 8s 25ms/step - accuracy: 0.9901 - loss: 0.0351 - val_accuracy: 0.8684 - val_loss: 0.6350
Epoch 7/10
313/313 ————— 8s 26ms/step - accuracy: 0.9921 - loss: 0.0310 - val_accuracy: 0.8662 - val_loss: 0.5906
Epoch 8/10
313/313 ————— 7s 24ms/step - accuracy: 0.9924 - loss: 0.0285 - val_accuracy: 0.8656 - val_loss: 0.6997
Epoch 9/10
313/313 ————— 8s 25ms/step - accuracy: 0.9927 - loss: 0.0263 - val_accuracy: 0.8666 - val_loss: 0.7230
Epoch 10/10
313/313 ————— 7s 23ms/step - accuracy: 0.9934 - loss: 0.0240 - val_accuracy: 0.8605 - val_loss: 0.6820

```

```

1 y_pred_d1 = (model.predict(X_test_pad) > 0.5).astype("int32")
2
3 print("Deep Learning Model Evaluation:\n")
4 print(classification_report(y_test_enc, y_pred_d1))

```

313/313 ————— 2s 7ms/step
 Deep Learning Model Evaluation:

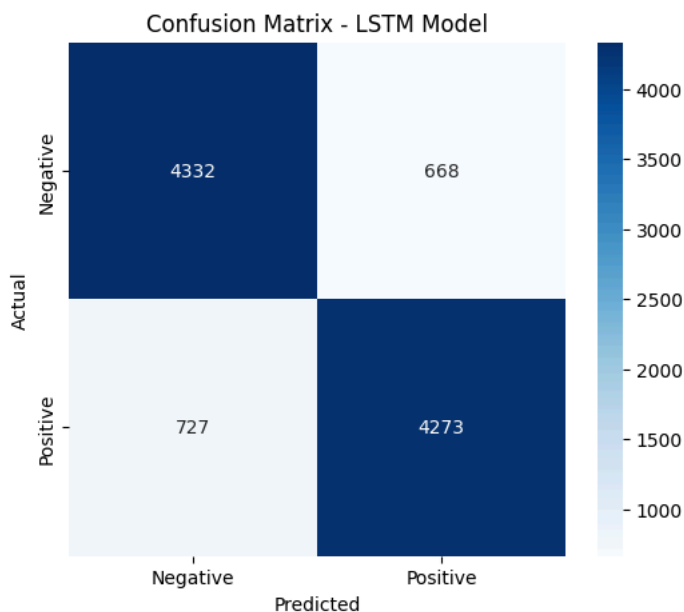
	precision	recall	f1-score	support
0	0.86	0.87	0.86	5000
1	0.86	0.85	0.86	5000

accuracy			0.86	10000
macro avg	0.86	0.86	0.86	10000
weighted avg	0.86	0.86	0.86	10000

```

1 cm_dl = confusion_matrix(y_test_enc, y_pred_dl)
2
3 plt.figure(figsize=(6,5))
4 sns.heatmap(cm_dl, annot=True, fmt="d", cmap="Blues", xticklabels=["Negative","Positive"], ytickl
5 plt.xlabel("Predicted")
6 plt.ylabel("Actual")
7 plt.title("Confusion Matrix - LSTM Model")
8 plt.show()

```



✓ Transformer Model (DistilBERT)

```

1 !pip install transformers datasets
2
3 from transformers import DistilBertTokenizerFast, TFDistilBertForSequenceClassification
4 from transformers import DataCollatorWithPadding
5 from transformers import Trainer, TrainingArguments
6 import tensorflow as tf

```

[Show hidden output](#)

tokenize

```

1 tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")
2
3 X_train_enc = tokenizer(list(X_train), truncation=True, padding=True, max_length=256, return_tensors="tf")
4 X_test_enc = tokenizer(list(X_test), truncation=True, padding=True, max_length=256, return_tensors="tf")
5
6 print("Train Encoded shape:", X_train_enc["input_ids"].shape)
7 print("Test Encoded shape:", X_test_enc["input_ids"].shape)

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 793B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 503kB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 9.90MB/s]
config.json: 100% 483/483 [00:00<00:00, 10.5kB/s]
TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or pinn
Train Encoded shape: (40000, 256)
Test Encoded shape: (10000, 256)

```

label encode

```

1 y_train_enc_tf = np.array(le.fit_transform(y_train))
2 y_test_enc_tf = np.array(le.transform(y_test))

```

model (Binary classification)

PyTorch weight কে TF এ কনভার্ট করবে

```

1 from transformers import TFDistilBertForSequenceClassification
2
3 bert_model = TFDistilBertForSequenceClassification.from_pretrained(
4     "distilbert-base-uncased",
5     num_labels=2,
6     from_pt=True
7 )

```

```

pytorch_model.bin: 100% 268M/268M [00:05<00:00, 49.8MB/s]
Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFDistilBertForSequenceClassification: ['vocab_p
- This IS expected if you are initializing TFDistilBertForSequenceClassification from a PyTorch model trained on another task or wi
- This IS NOT expected if you are initializing TFDistilBertForSequenceClassification from a PyTorch model that you expect to be exa
Some weights or buffers of the TF 2.0 model TFDistilBertForSequenceClassification were not initialized from the PyTorch model and a
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```

```

1 from transformers import create_optimizer
2
3 steps_per_epoch = len(y_train_enc_tf) // 16
4 num_train_steps = steps_per_epoch * 2 # epochs=2
5 optimizer, lr_schedule = create_optimizer(
6     init_lr=5e-5,
7     num_warmup_steps=0,
8     num_train_steps=num_train_steps
9 )
10
11 loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
12 bert_model.compile(optimizer=optimizer, loss=loss, metrics=["accuracy"])
13

```

```

1 history_bert = bert_model.fit(
2     x=dict(X_train_enc),
3     y=y_train_enc_tf,
4     validation_data=(dict(X_test_enc), y_test_enc_tf),
5     epochs=2,
6     batch_size=16
7 )

```

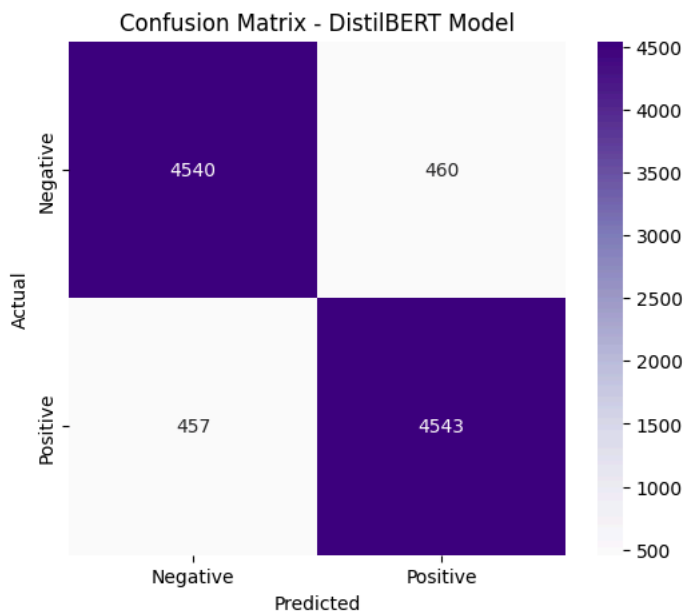
```
Epoch 1/2
2500/2500 [=====] - 1204s 481ms/step - loss: 0.2884 - accuracy: 0.8809 - val_loss: 0.2383 - val_accuracy:
Epoch 2/2
2500/2500 [=====] - 1142s 457ms/step - loss: 0.1360 - accuracy: 0.9505 - val_loss: 0.2505 - val_accuracy:
```

```
1 y_pred_logits = bert_model.predict(dict(X_test_enc)).logits
2 y_pred_bert = np.argmax(y_pred_logits, axis=1)
3
4 print("DistilBERT Evaluation:\n")
5 print(classification_report(y_test_enc_tf, y_pred_bert))
```

```
313/313 [=====] - 92s 288ms/step
DistilBERT Evaluation:
```

	precision	recall	f1-score	support
0	0.91	0.91	0.91	5000
1	0.91	0.91	0.91	5000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

```
1 cm_bert = confusion_matrix(y_test_enc_tf, y_pred_bert)
2
3 plt.figure(figsize=(6,5))
4 sns.heatmap(cm_bert, annot=True, fmt="d", cmap="Purples", xticklabels=["Negative","Positive"], yt
5 plt.xlabel("Predicted")
6 plt.ylabel("Actual")
7 plt.title("Confusion Matrix - DistilBERT Model")
8 plt.show()
```



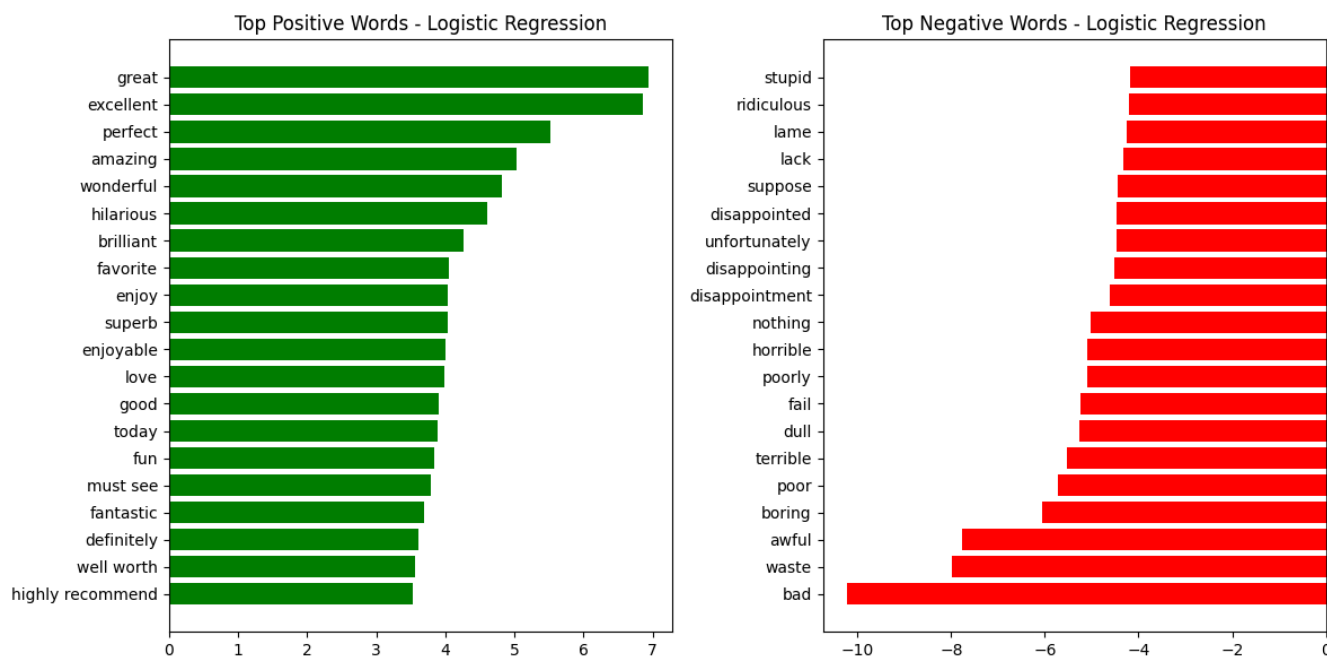
** Lime for Logistic Regression / LSTM**

```
1 feature_names = tfidf.get_feature_names_out()
2 coefficients = log_reg.coef_[0]
3
4 top_pos = np.argsort(coefficients)[-20:]
5 top_neg = np.argsort(coefficients)[:20]
6
7 plt.figure(figsize=(12,6))
8
9 plt.subplot(1,2,1)
```

```

10 plt.barh(feature_names[top_pos], coefficients[top_pos], color="green")
11 plt.title("Top Positive Words - Logistic Regression")
12
13 plt.subplot(1,2,2)
14 plt.barh(feature_names[top_neg], coefficients[top_neg], color="red")
15 plt.title("Top Negative Words - Logistic Regression")
16
17 plt.tight_layout()
18 plt.show()
19

```



```

1 from lime.lime_text import LimeTextExplainer
2 import numpy as np
3
4 def bert_predict_proba_texts(texts):
5     enc = tokenizer(
6         list(texts),
7         truncation=True,
8         padding=True,
9         max_length=256,
10        return_tensors="tf"
11    )
12    logits = bert_model(enc).logits
13    probs = tf.nn.softmax(logits, axis=1).numpy()
14    return probs
15
16 explainer = LimeTextExplainer(class_names=["NEG", "POS"])
17
18 sample_idx = 0
19 sample_text = X_test.iloc[sample_idx]
20
21 exp = explainer.explain_instance(
22     sample_text,
23     bert_predict_proba_texts,
24     num_features=10,
25     num_samples=2000
26 )

```

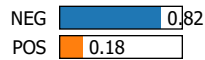
```

27
28 # Show explanation
29 print("Text:", sample_text)
30 print("True label:", y_test.iloc[sample_idx])
31 exp.show_in_notebook(text=True)
32

```

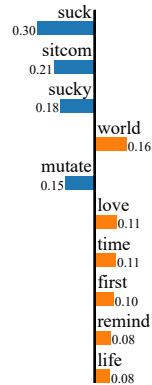
Text: yes mtv really way market daria start clever teenage angst comment everything suck make viewer feel well sucky teenage life s
 True label: negative

Prediction probabilities



NEG

POS



Text with highlighted words

yes mtv really way market daria start clever teenage angst
 comment everything suck make viewer feel well sucky
 teenage life sitcom mutate deal problem charade use watch
 daria time love sit watch call movie wonder point daria tell
 lead life college excuse point daria make every episode like
 long matter rest sick sad world think entire thing remind
 scene reality bite movie channel show documentary first time

```

1 logreg_report = classification_report(y_test, y_pred, output_dict=True)
2
3 lstm_report = classification_report(y_test_enc, y_pred_dl, output_dict=True)
4
5 bert_report = classification_report(y_test_enc_tf, y_pred_bert, output_dict=True)

```

```

1 # Create a dataframe for comparison
2 comparison_df = pd.DataFrame({
3     "Model": ["Logistic Regression (TF-IDF)", "LSTM", "DistilBERT"],
4     "Accuracy": [
5         logreg_report["accuracy"],
6         lstm_report["accuracy"],
7         bert_report["accuracy"]
8     ],
9     "Precision_Pos": [
10         logreg_report["positive"]["precision"],
11         lstm_report["1"]["precision"], # LSTM label 1 = positive
12         bert_report["1"]["precision"]
13     ],
14     "Recall_Pos": [
15         logreg_report["positive"]["recall"],
16         lstm_report["1"]["recall"],
17         bert_report["1"]["recall"]
18     ],
19     "F1_Pos": [
20         logreg_report["positive"]["f1-score"],
21         lstm_report["1"]["f1-score"],
22         bert_report["1"]["f1-score"]
23     ],
24     "Precision_Neg": [
25         logreg_report["negative"]["precision"],
26         lstm_report["0"]["precision"],
27         bert_report["0"]["precision"]
28     ],
29     "Recall_Neg": [
30         logreg_report["negative"]["recall"],
31         lstm_report["0"]["recall"],
32         bert_report["0"]["recall"]
33     ],

```

```
34     "F1_Neg": [  
35         logreg_report["negative"]["f1-score"],  
36         lstm_report["0"]["f1-score"],  
37         bert_report["0"]["f1-score"]  
38     ]  
39 })  
40  
41 comparison_df  
42
```

	Model	Accuracy	Precision_Pos	Recall_Pos	F1_Pos	Precision_Neg	Recall_Neg	F1_Neg	
0	Logistic Regression (TF-IDF)	0.8912	0.882629	0.9024	0.892405	0.900164	0.8800	0.889968	
1	LSTM	0.8605	0.864805	0.8546	0.859672	0.856296	0.8664	0.861318	 